```
#include "FEM_Solver.h"

FEM_Solver::FEM_Solver(const Mesh& me) : mesh(me) {}

void FEM_Solver::build_FEM()
{
     triplets_.clear();

     const auto& faces = mesh.facez();
     const int nVerts =
static_cast<int>(mesh.verticez().size());

     for (int f = 0; f < static_cast<int>(faces.size()); ++f)
     {
          handle_triangle(f);
     }

     A_.resize(nVerts, nVerts);
     A_.setFromTriplets(triplets_.begin(), triplets_.end());
}

void FEM_Solver::solve()
{
     build_FEM();
     build_RHS();

     Eigen::SparseLU<Eigen::SparseMatrix<std::complex<double>>>
solver;
     solver.compute(A_);
     if (solver.info() != Eigen::Success) {
          std::cerr << "Factorization failed\n";
          return;
     }

     u = solver.solve(b);
     if (solver.info() != Eigen::Success) {
          std::cerr << "Solve failed\n";
          return;
     }

     std::cout << "Solution u:\n" << u << "\n";
```

```
}
void FEM_Solver::handle_triangle(int faceIndex)
{

     const auto& faces = mesh.facez();
     const auto& verts = mesh.verticez();

     const triangle& T = faces[faceIndex];
     int ia = T.a;
     int ib = T.b;
     int ic = T.c;

     const Vec3& xa = verts[ia];
     const Vec3& xb = verts[ib];
     const Vec3& xc = verts[ic];

     Eigen::Vector3d a(xa.x, xa.y, xa.z);
     Eigen::Vector3d b(xb.x, xb.y, xb.z);
     Eigen::Vector3d c(xc.x, xc.y, xc.z);

     Eigen::Vector3d e1 = b - a;
     Eigen::Vector3d e2 = c - a;
     double area = triangle_area(e1, e2);

     Eigen::Matrix3d K_T = local_stiffness(e1, e2, area);
     Eigen::Matrix3d M_T = local_mass(xa, xb, xc, area);

     double V_T = potential_on_triangle(xa, xb, xc);
     Eigen::Matrix3d W_T = V_T * M_T;

     double coeff = hbar * hbar / (2 * m);
     Eigen::Matrix3d A_T = coeff * K_T + W_T;
     if (counter == 0) { std::cout << A_T; }

     // 3. assemble into global sparse A via triplets
     int ids[3] = { ia, ib, ic };

     for (int i = 0; i < 3; ++i) {
          for (int j = 0; j < 3; ++j) {
               std::complex<double> val = A_T(i, j);
               if (std::abs(val) > 0.0) {
```

```
                    triplets_.emplace_back(ids[i], ids[j], val);
                }
            }
        }
}

double inline FEM_Solver::triangle_area(const Eigen::Vector3d&
e1, const Eigen::Vector3d& e2) const
{
    return 0.5 * e1.cross(e2).norm();
}

Eigen::Matrix3d FEM_Solver::local_mass(const Vec3& xa, const
Vec3& xb, const Vec3& xc, double area) const
{
    Eigen::Matrix3d M_T;
    M_T << 2, 1, 1,
         1, 2, 1,
         1, 1, 2;
    M_T *= area / 12.0;
    return M_T;
}

double FEM_Solver::potential_on_triangle(const Vec3& xa, const
Vec3& xb, const Vec3& xc) const
{
    // barycenter on sphere
    Vec3 b{ (xa.x + xb.x + xc.x) / 3.0,
              (xa.y + xb.y + xc.y) / 3.0,
              (xa.z + xb.z + xc.z) / 3.0 };

    double len = std::sqrt(b.x * b.x + b.y * b.y + b.z * b.z);
    b.x /= len; b.y /= len; b.z /= len;
    //compute V_T
    double k = 1;
    double z = std::max(-1.0, std::min(1.0, b.z));
    double d = std::acos(z);// unit sphere, north pole center
    return 0.5 * k * d * d;
}

void FEM_Solver::build_RHS()
```

```
{
     const auto& faces = mesh.facez();
     const auto& verts = mesh.verticez();
     const int nVerts = static_cast<int>(verts.size());

     b = Eigen::VectorXcd::Zero(nVerts);

     for (int f = 0; f < static_cast<int>(faces.size()); ++f) {
          const triangle& T = faces[f];

          int ia = T.a;
          int ib = T.b;
          int ic = T.c;

          const Vec3& xa = verts[ia];
          const Vec3& xb = verts[ib];
          const Vec3& xc = verts[ic];

          Eigen::Vector3d a(xa.x, xa.y, xa.z);
          Eigen::Vector3d bvec(xb.x, xb.y, xb.z);
          Eigen::Vector3d c(xc.x, xc.y, xc.z);

          Eigen::Vector3d e1 = bvec - a;
          Eigen::Vector3d e2 = c - a;
          double area = triangle_area(e1, e2);

          Eigen::Vector3cd FT = local_rhs(xa, xb, xc, area);

          b(ia) += FT(0);
          b(ib) += FT(1);
          b(ic) += FT(2);
     }
}

std::complex<double> FEM_Solver::source_at_point(const Vec3& x)
const
{
     double phi = std::atan2(x.y, x.x);

     double z = std::max(-1.0, std::min(1.0, x.z));
```

```
     double d = std::acos(z); // north pole center on unit
sphere

     std::complex<double> phase =
std::exp(std::complex<double>(0.0, ell_ * phi));
     double envelope = std::exp(-(d * d) / (sigma_ * sigma_));

     return A0_ * phase * envelope;
}

Eigen::Vector3cd FEM_Solver::local_rhs(const Vec3& xa, const
Vec3& xb, const Vec3& xc, double area) const
{
      Eigen::Matrix3d MT = local_mass(xa, xb, xc, area);

    Vec3 b{
        (xa.x + xb.x + xc.x) / 3.0,
        (xa.y + xb.y + xc.y) / 3.0,
        (xa.z + xb.z + xc.z) / 3.0
    };

    double len = std::sqrt(b.x * b.x + b.y * b.y + b.z * b.z);
    b.x /= len;
    b.y /= len;
    b.z /= len;

    std::complex<double> fT = source_at_point(b);

    Eigen::Vector3d ones(1.0, 1.0, 1.0);
     return fT * (MT * ones).cast<std::complex<double>>();
}

Eigen::Matrix3d FEM_Solver::local_stiffness(const
Eigen::Vector3d& e1, const Eigen::Vector3d& e2, double area)
const
{
     Eigen::Matrix2d G;
     G << e1.dot(e1), e1.dot(e2),
          e2.dot(e1), e2.dot(e2);

     Eigen::Matrix2d Ginv = G.inverse();
```

```
    Eigen::Matrix<double, 3, 2> R;
    R << -1.0, -1.0,
         1.0, 0.0,
         0.0, 1.0;

    Eigen::Matrix3d K = area * R * Ginv * R.transpose();
    if (counter == 0) { std::cout << K; }
    return K;
}
```